BlockSupply
Decentralized Supply Cloud

AWS Cloud Computing Project — Documentation

Supply Chain Management Dashboard demonstrating AWS cloud concepts: EC2, RDS (MariaDB), S3, CloudWatch, IAM, and VPC.

Submitted: 20 June 2026

Contents

1. Project Overview

BlockSupply is a Supply Chain Management Dashboard built to demonstrate practical use of core AWS cloud services in a real, working application rather than in isolation. The application is a Flask web app supporting three user roles — Admin, Manager, and Operational Staff — and covers inventory management, shipment tracking, workflow management with approval chains, and a reporting dashboard with data export to S3.

The goal of the project is twofold: deliver a usable internal tool, and show hands-on configuration of EC2, RDS, S3, CloudWatch, IAM, and VPC working together as a single deployed system.

2. Application Features & User Roles

User Roles

Role	Permissions
Admin	Full access — manage inventory, shipments, tasks, approve/reject workflow requests, export reports
Manager	Add/update inventory, create shipments, assign tasks (routed to Admin for approval), view reports, export
Operational Staff	View inventory/shipments/tasks, update product quantities and shipment/task status

Modules

Inventory Management — Add products, update quantity, warehouse tracking with low-stock alerts

Shipment Tracking — Shipment status (Pending / In Transit / Delivered / Cancelled), source/destination

Workflow Management — Task assignment with due dates; manager-created tasks route through an admin approval chain

Reporting Dashboard — Inventory metrics by warehouse, shipment status analytics, low-stock alerts, CSV export to S3

3. Cloud Architecture

The application is deployed inside a custom VPC with a public subnet hosting the EC2 instance and a private subnet hosting the RDS database. S3 sits outside the VPC and is accessed through an IAM instance role attached to EC2 — no access keys are stored in code or configuration files. CloudWatch monitors both infrastructure metrics (CPU, network on EC2 and RDS) and custom application metrics pushed via boto3.

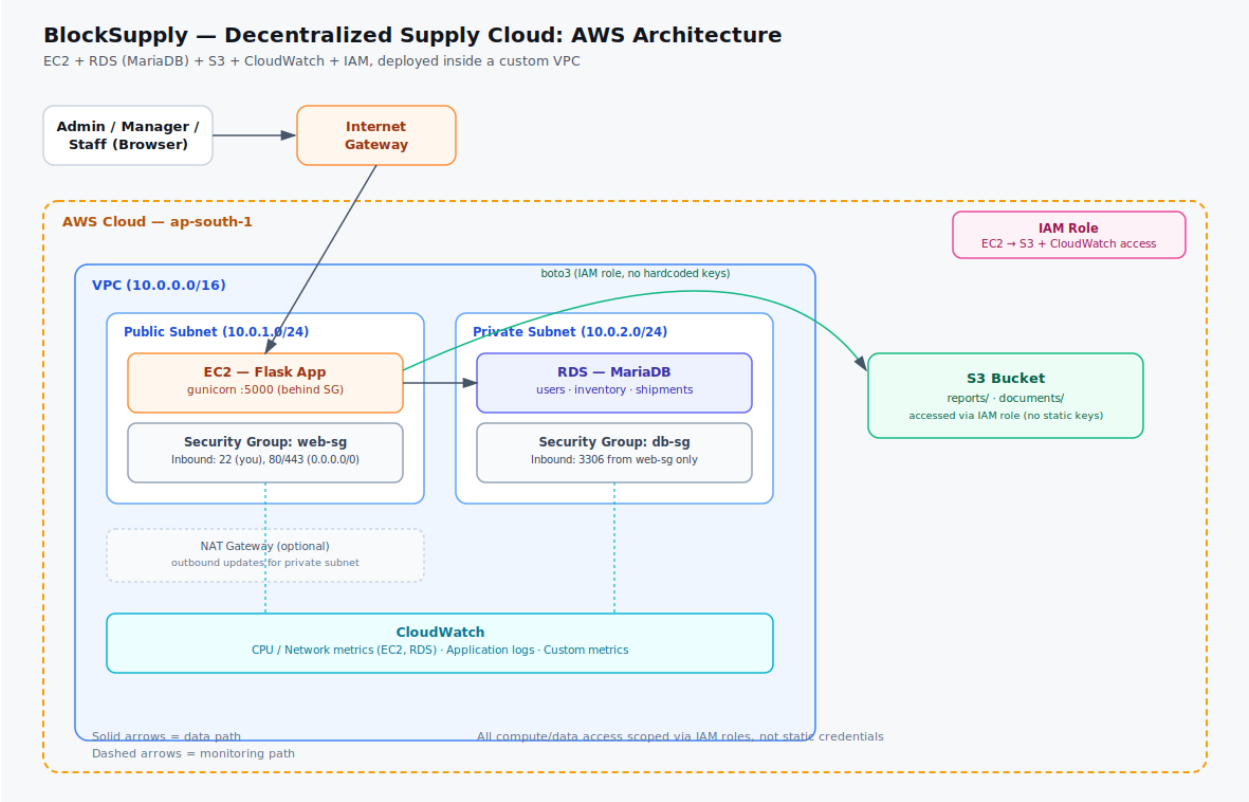

Figure 1 — BlockSupply AWS architecture

Network Design

Component	Detail
VPC CIDR	10.0.0.0/16
Public Subnet	10.0.1.0/24 — hosts EC2 (Flask app)
Private Subnet	10.0.2.0/24 — hosts RDS (MariaDB)
web-sg	Inbound: 22 (admin IP), 80/443 (public)
db-sg	Inbound: 3306 from web-sg only — RDS has no public access

4. Database Design

The schema is implemented in MariaDB on Amazon RDS. Six core tables capture users, warehouses, products, shipments, tasks, and the approval chain that links tasks to admin sign-off.

users

Stores Admin/Manager/Staff accounts with hashed passwords

```
id, username, password_hash, role, created_at
```

warehouses

Physical storage locations

```
id, name, location
```

products

Inventory items, linked to a warehouse

```
id, name, sku, quantity, reorder_level, warehouse_id, updated_at
```

shipments

Shipment records with status lifecycle

```
id, product_id, quantity, source, destination, status, created_at, updated_at
```

tasks

Workflow tasks assigned to users

```
id, title, description, assigned_to, created_by, status, due_date, created_at
```

approval_chains

Approval records linking tasks to an approving admin

```
id, task_id, approver_id, status, comment, decided_at
```

Relationships

products.warehouse_id → warehouses.id | shipments.product_id → products.id | tasks.assigned_to / created_by → users.id | approval_chains.task_id → tasks.id, approval_chains.approver_id → users.id

Full DDL is provided in schema.sql in the project repository.

5. AWS Component Implementation

Service	Role in this project
EC2	Hosts the Flask application via gunicorn, managed as a systemd service for auto-restart
RDS (MariaDB)	Stores users, inventory, and shipment data; Single-AZ db.t3.micro in the private subnet
S3	Stores exported CSV inventory reports under reports/ and supports documents/ for future use
CloudWatch	Monitors EC2/RDS CPU and network automatically; receives custom app metrics (UserLogin, ReportExpo
IAM	An EC2 instance role grants S3 and CloudWatch access — no access keys are hardcoded anywhere in th
VPC	Public subnet for EC2 (internet-facing), private subnet for RDS (isolated from the internet, reachable only

6. Deployment Summary

Full step-by-step instructions are in deploy/DEPLOYMENT_GUIDE.md in the repository. At a high level, the deployment order was:

1. Create the VPC with one public and one private subnet, plus web-sg and db-sg security groups
2. Provision the RDS MariaDB instance in the private subnet (db.t3.micro, Single-AZ)
3. Create the S3 bucket for reports/documents with public access blocked
4. Create an IAM role granting the EC2 instance S3 and CloudWatch access
5. Launch the EC2 instance in the public subnet with the IAM role attached
6. Clone the repository, install dependencies, configure .env with the RDS endpoint and S3 bucket name
7. Run flask seed-db to create tables and demo users
8. Run the app via systemd (gunicorn) for a stable, auto-restarting deployment
9. Verify CloudWatch is receiving EC2/RDS metrics and custom application metrics

7. Pricing Analysis

This architecture is fully covered by the AWS Free Tier for the first 12 months on a new account. The table below also shows standard on-demand pricing (region: ap-south-1) for reference once the free tier expires.

Component	Spec	Free Tier	On-Demand (after Free Tier)
EC2	t2.micro, 24/7	$0	≈ $9–10/mo
EBS	20 GB gp2	$0	≈ $2/mo
RDS (MariaDB)	db.t3.micro, Single-AZ, 20GB	$0	≈ $24–26/mo
S3	<1 GB, low requests	$0	≈ $0.03/GB-mo + requests
CloudWatch	Default + custom metrics	$0	≈ $0.30/metric beyond free 10
Data Transfer Out	Demo-level	$0	$0.09/GB beyond 100GB free
IAM / VPC	Roles, no NAT Gateway	$0	$0

Estimated total: $0/month on Free Tier · ≈ $33–36/month on standard on-demand pricing, dominated by RDS instance hours. No NAT Gateway is used (the private subnet doesn't need outbound internet), which avoids a common ~$32+/month cost on student AWS bills.

Prices are approximate and change periodically — see calculator.aws for an exact, current quote.

8. Screenshots

Insert screenshots here after deployment, in this order:

- EC2 instance running (console)
- RDS instance available, with endpoint visible
- S3 bucket showing an uploaded report
- CloudWatch — EC2 CPU/Network graph
- CloudWatch — custom metric (UserLogin or ReportExported)
- Application running in browser: Dashboard, Inventory, Shipments, Workflow, Reports
- Security group inbound rules (web-sg and db-sg)

[Placeholder — replace this page with actual screenshots taken from your AWS console and running application before final submission]

9. Deliverables Checklist

- Cloud Architecture Diagram — deploy/architecture-diagram.png
- Flask Source Code — app.py, models.py, config.py, templates/, static/
- Database Design — schema.sql + Section 4 of this document
- Pricing Analysis — Section 7 of this document
- CloudWatch Screenshots — Section 8 of this document
- Documentation PDF — this document
- GitHub Repository — pushed with README.md as the entry point